

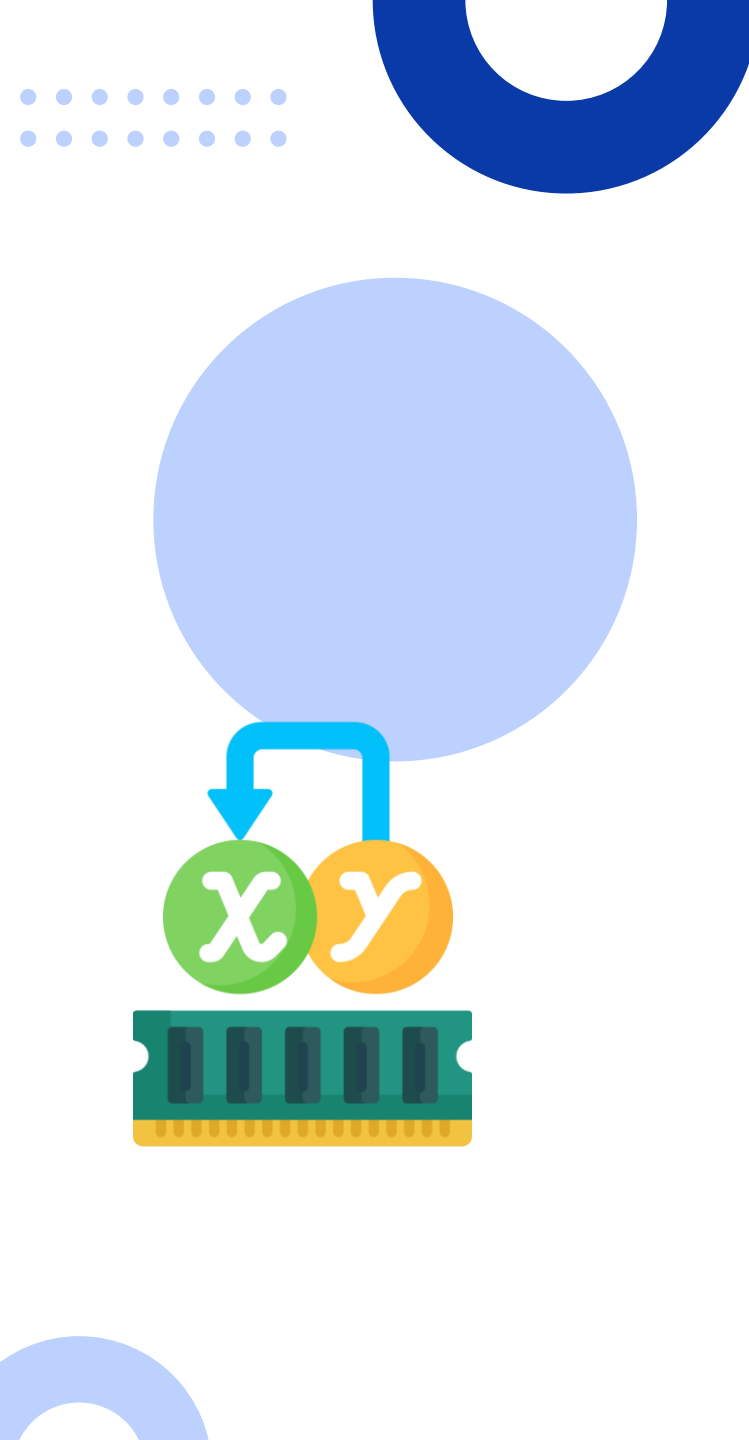
บทที่ 2

ตัวแปร และชนิดข้อมูล

Variable and Datatype

อาจารย์ผู้สอน:

- ▶ อ. ดร.พงษ์ศร จันทรียอย
วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น

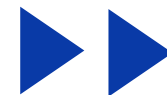


วัตถุประสงค์การเรียนรู้

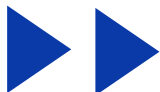
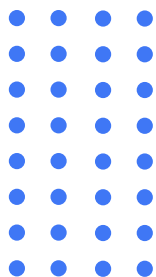
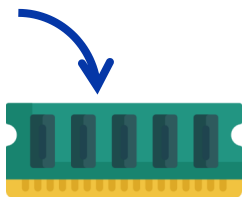
- เพื่อให้ผู้เรียนเข้าใจแนวคิดและความสำคัญของตัวแปร
- เพื่อให้ผู้เรียนสามารถประกาศตัวแปรและกำหนดค่าในภาษา Python ได้อย่างถูกต้อง
- เพื่อให้ผู้เรียนเข้าใจกฎการตั้งชื่อตัวแปรและรูปแบบการตั้งชื่อที่เหมาะสม
- เพื่อให้ผู้เรียนเข้าใจชนิดข้อมูลพื้นฐาน ชนิดข้อมูลเชิงประกอบ
- เพื่อให้ผู้เรียนสามารถเลือกใช้ชนิดข้อมูลที่เหมาะสมกับปัญหา

เนื้อหา

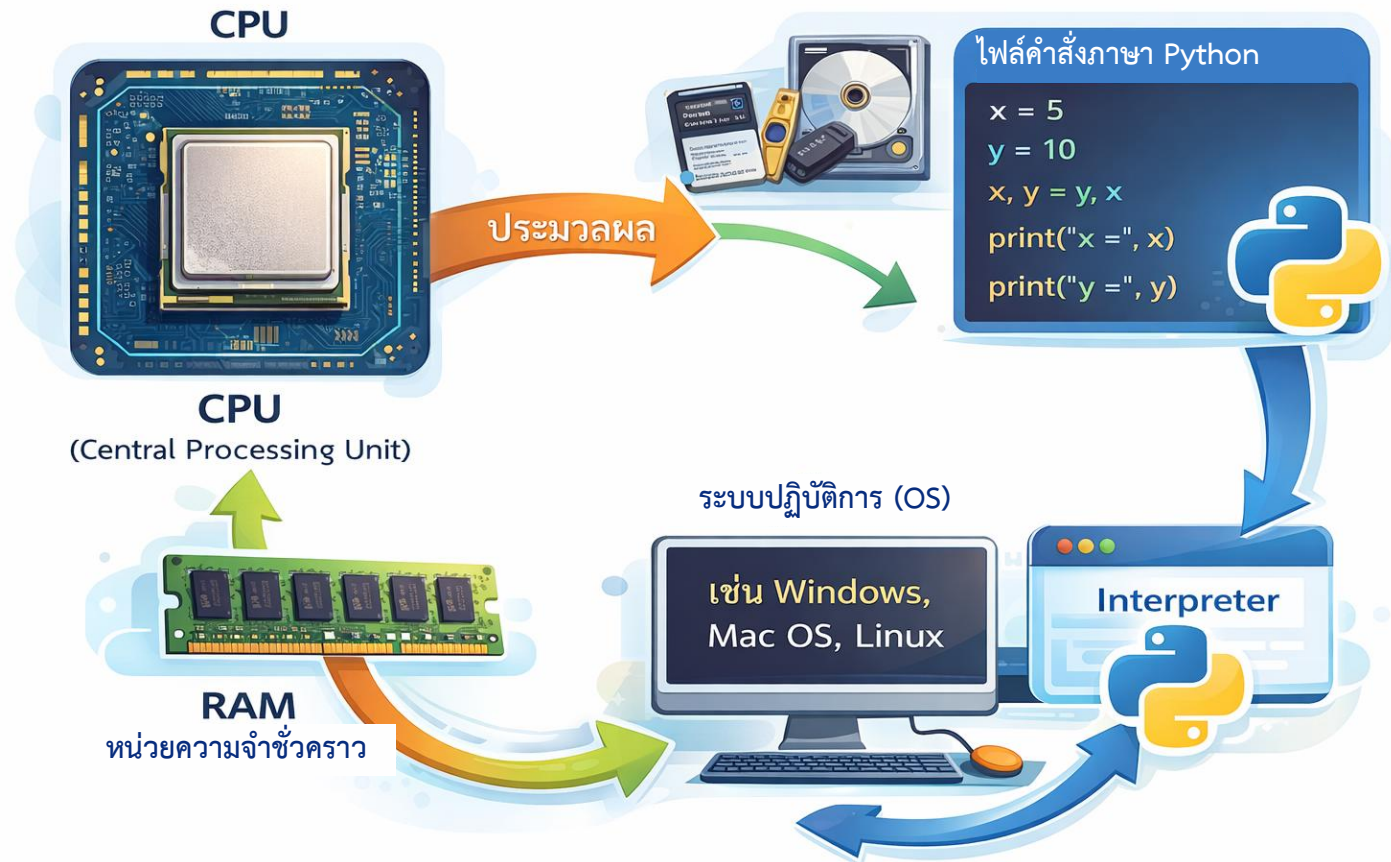
01. รู้จักและใช้งานตัวแปร
02. การตั้งชื่อตัวแปร
03. คำสงวน (Reserved word, Keyword)
04. การตรวจสอบชนิดตัวแปร
05. ชนิดข้อมูล



รู้จัก และใช้งานตัวแปร

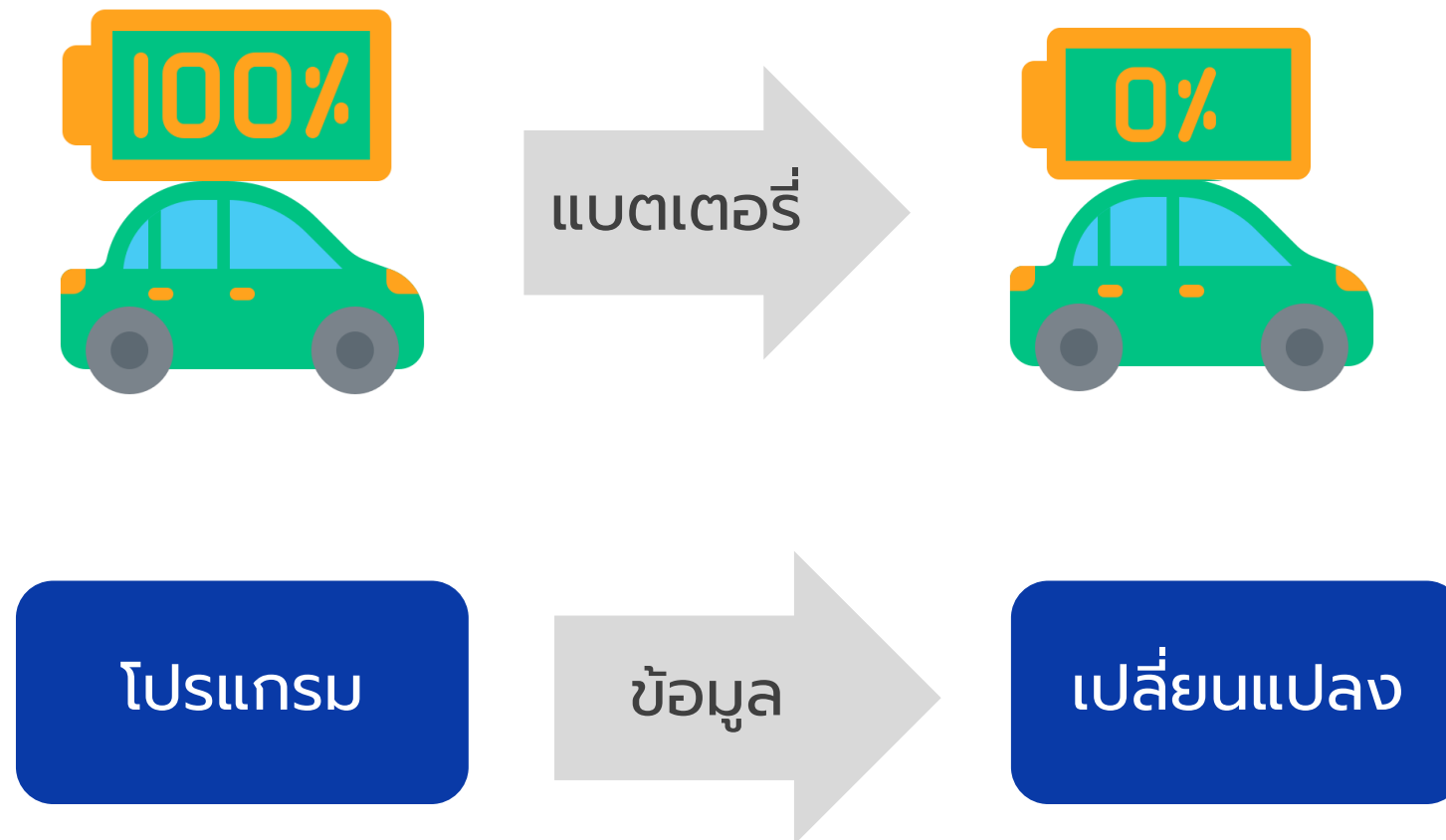


การทำงานของโปรแกรม



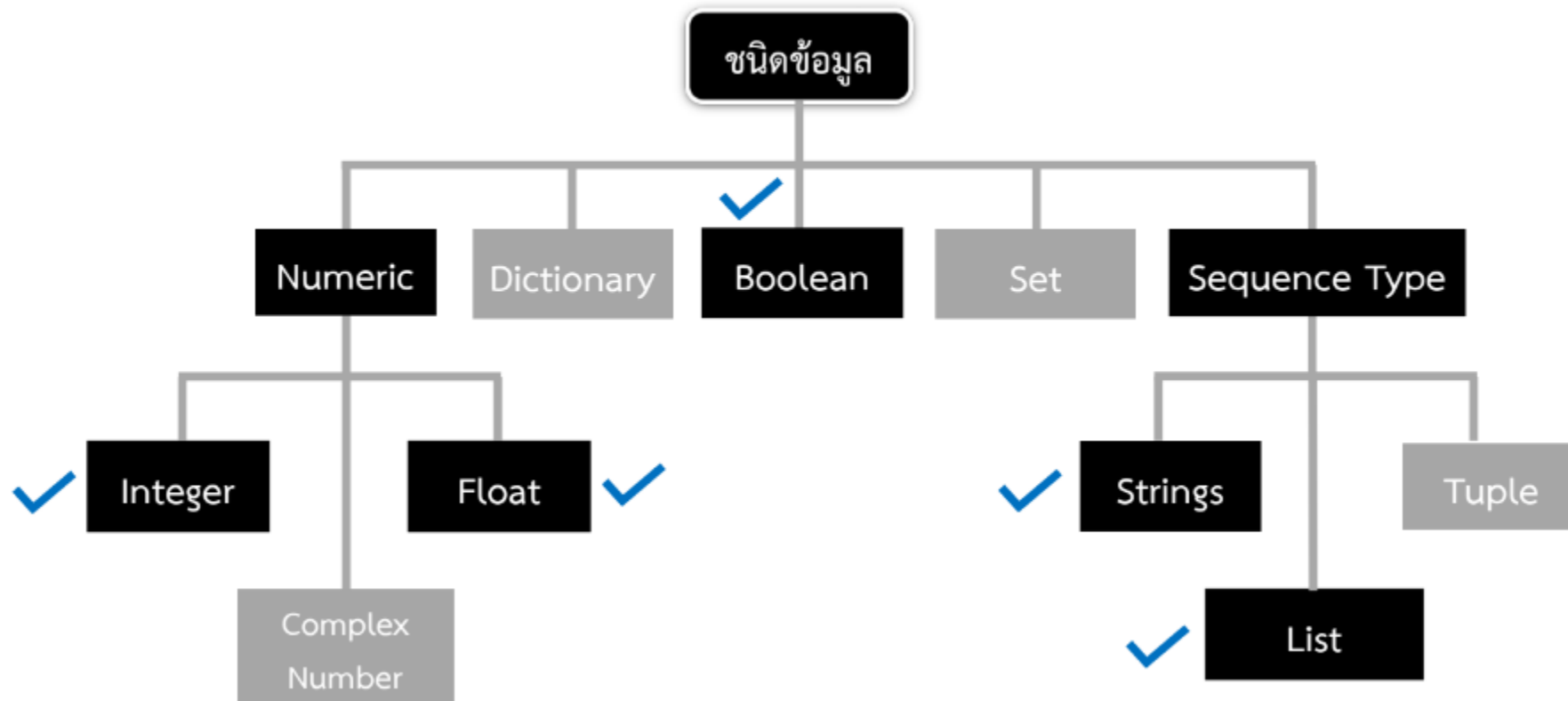
ข้อมูลในการเขียนโปรแกรมภาษา Python

- สิ่งใดมีการเปลี่ยนแปลง ?

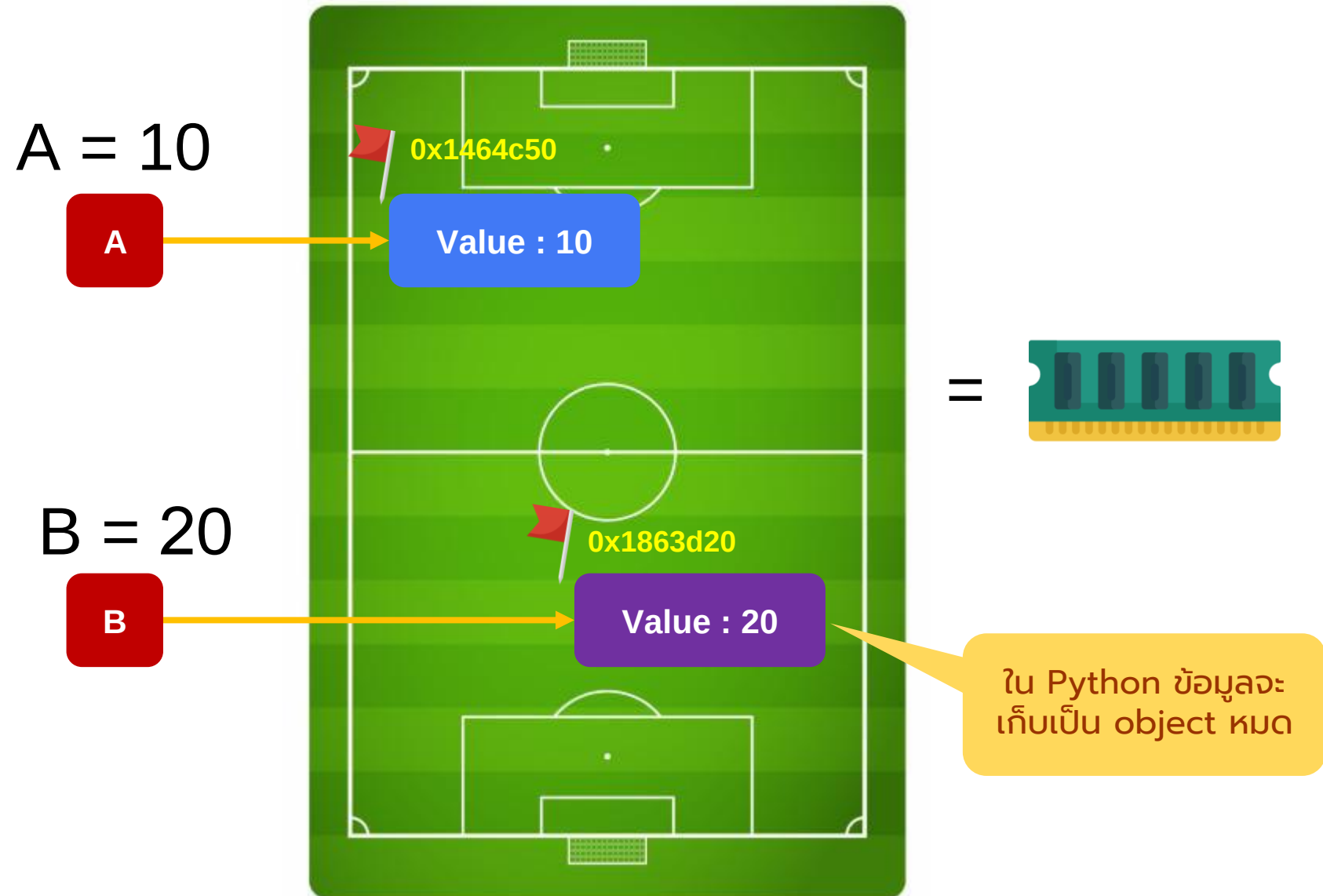


ชนิดข้อมูล (Data Type)

- ชนิดข้อมูลที่ใช้งานงานบ่อย



ตัวแปร



รู้จักและใช้งานตัวแปร

- **ตัวแปร**

- คือ **ชื่อที่กำหนดขึ้นสำหรับใช้อ้างถึงค่าที่เก็บในหน่วยความจำ**ตอนที่โปรแกรมทำงาน
- ข้อมูลที่เก็บในตัวแปรสามารถ**เปลี่ยนแปลง**ตอนที่โปรแกรมทำงาน
- การตั้งชื่อตัวแปรจะช่วยให้เรา**เขียนโปรแกรมได้ง่ายขึ้น**
- โดยสามารถ**อ้างถึงข้อมูลโดยใช้ชื่อตัวแปรได้เลย** ซึ่งสะดวกและ**จดจำได้ง่าย**

รู้จักและใช้งานตัวแปร

- ถ้าไม่มีตัวแปร

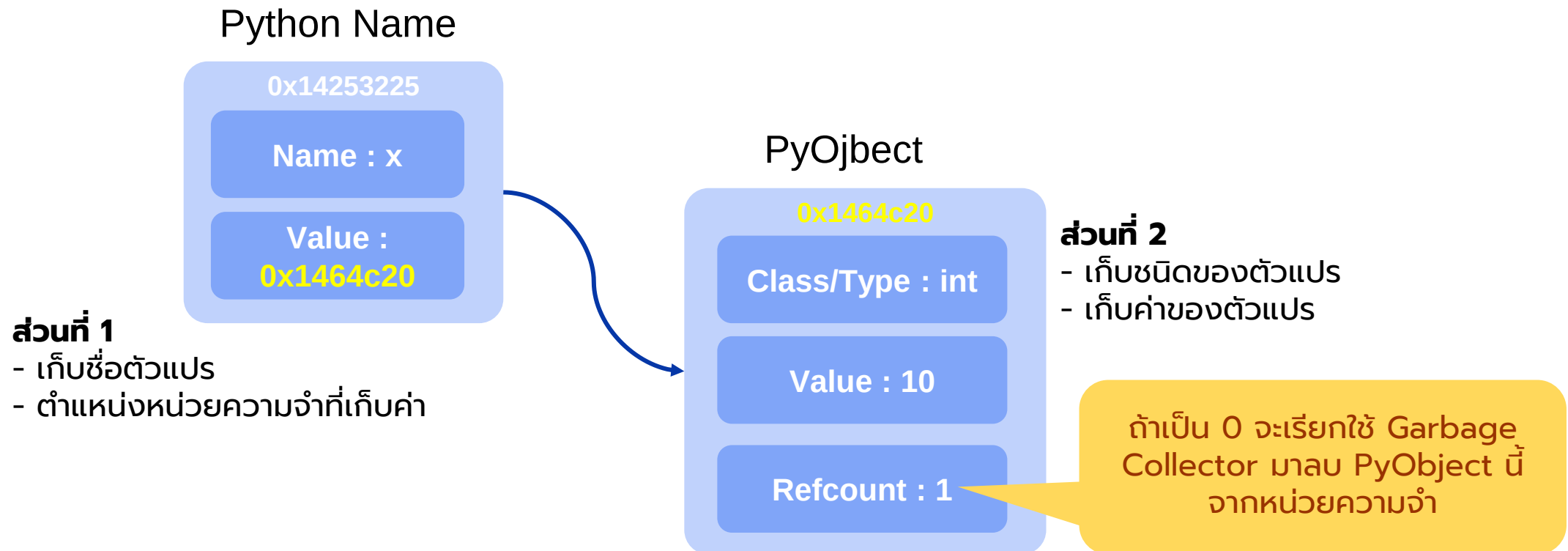
- เราต้องการเก็บเลข 10 และ 20 แล้วนำมาบวกกัน ในหน่วยความจำเราจะต้องบอกว่า
 - นำตัวเลข 10 เก็บไว้ในตำแหน่งที่ **0x1464C20** (เลขฐานสิบหก) ของหน่วยความจำ
 - นำตัวเลข 20 เก็บไว้ในตำแหน่งที่ **0x1464C50** (เลขฐานสิบหก)
 - ดึงค่าในตำแหน่ง **0x1464C20** มาบวกกับค่าในตำแหน่ง **0x1464C50**

- ถ้ามีตัวแปร

- เราต้องการเก็บเลข 10 และ 20 แล้วนำมาบวกกัน ในหน่วยความจำเราจะต้องบอกว่า
 - กำหนดก่อนว่าให้ตัวแปร **A** อ้างอิงไปตำแหน่งที่ **0x1464C20** ของหน่วยความจำ
 - กำหนดก่อนว่าให้ตัวแปร **B** อ้างอิงไปตำแหน่งที่ **0x1464C50** ของหน่วยความจำ
 - นำ **A + B** ก็จะเป็นการนำค่าในตำแหน่งนั้นๆ มาบวกกัน

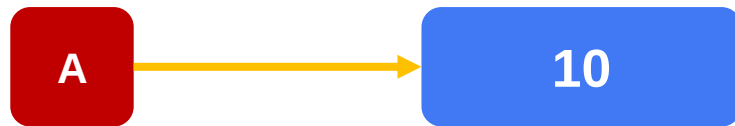
ตัวแปรในภาษา Python

$x = 10$

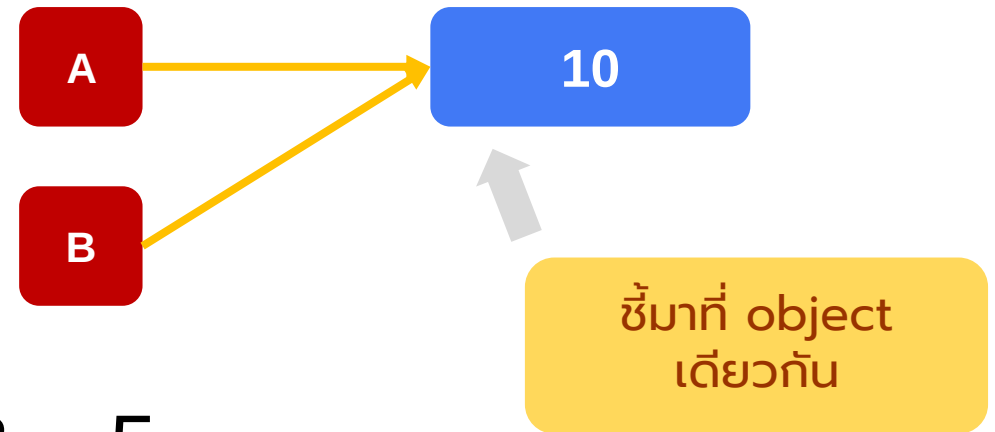


ตัวแปรในภาษา Python

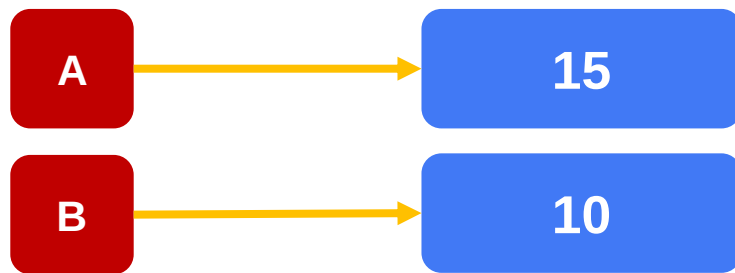
A = 10



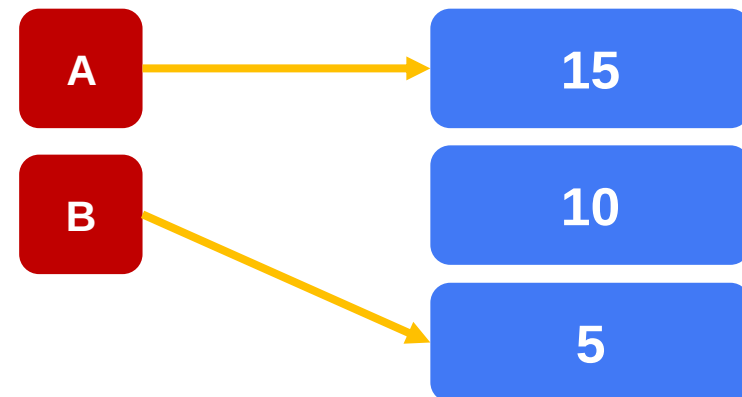
B = A



A = 15

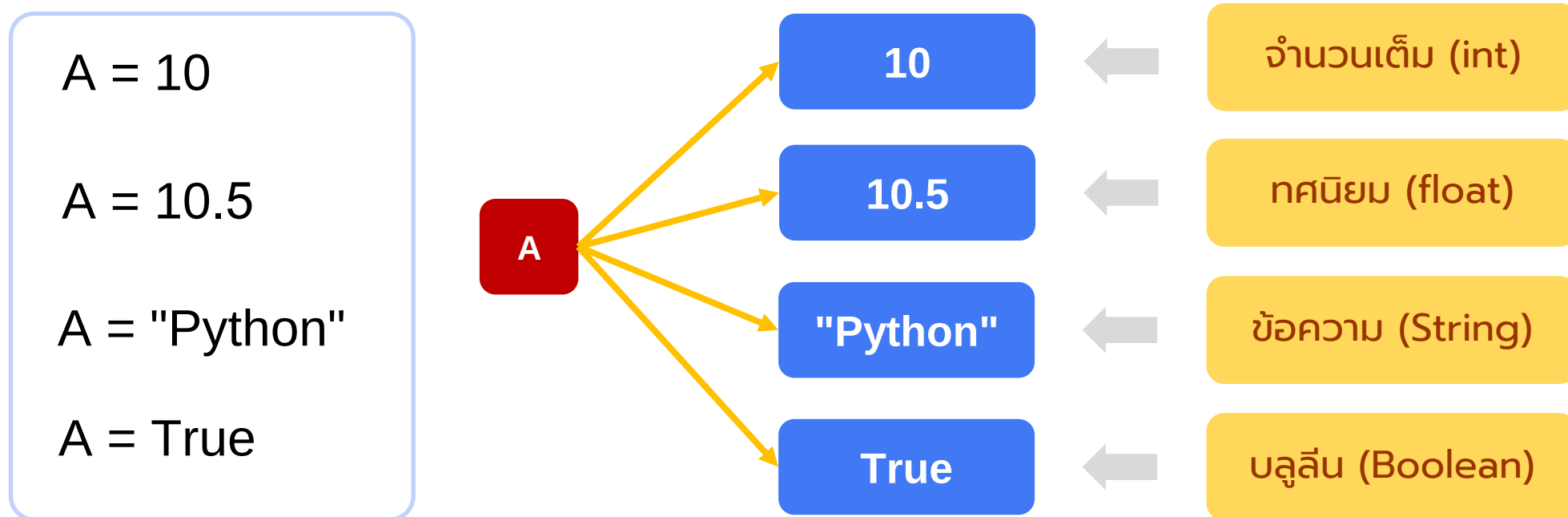


B = 5



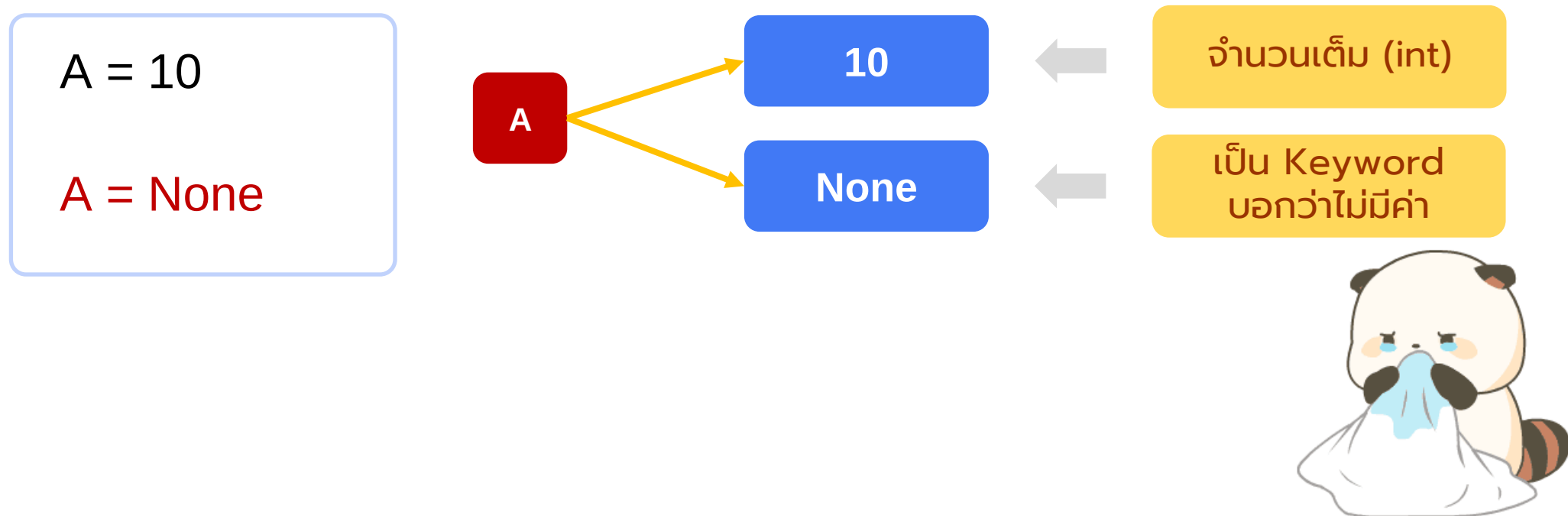
ชนิดตัวแปรในภาษา Python เป็น Dynamic Type

- ตัวแปรใน Python มีลักษณะเป็น Type infer language (Dynamic Type)
- ไม่ต้องกำหนดชนิดของตัวแปร Python จะรู้เองผ่านการกำหนดค่า



ชนิดตัวแปรในภาษา Python เป็น Dynamic Type

- ตัวแปรใน Python มีลักษณะเป็น Dynamic Type
- ไม่ต้องกำหนดชนิดของตัวแปร Python จะรู้เองผ่านการกำหนดค่า



กฎการตั้งชื่อตัวแปรในภาษา Python

- ตัวแปรต้องขึ้นต้นด้วย ตัวอักษร A-z หรือ _ (underscore) เท่านั้น
- ไม่สามารถขึ้นต้นด้วยตัวเลข number 0-9
- ห้ามมีช่องว่างหรือสัญลักษณ์พิเศษ (special symbols) เช่น !, @, #, \$, % ยกเว้น underscores (_) เท่านั้น
- มีลักษณะเป็น case-sensitive การใช้ตัวอักษรตัวพิมพ์ใหญ่พิมพ์เล็กต่างกัน เช่น `firstname`, `Firstname`, `FirstName` and `FIRSTNAME` จะถือว่าทุกตัวเป็นคนละตัวแปรกัน
- ห้ามใช้คำสงวน (Reserved words, Keyword) ตั้งเป็นชื่อตัวแปร

ตัวแปรใดตั้งชื่อผิดกฎบ้าง ?

- firstname
- first-name
- lastname
- age
- country
- city
- last_name
- capital_city
- _if
- year_2021
- year2021
- current_year_2021
- birth_year
- num1
- num2
- first@name
- first_name
- first\$name
- num-1
- last_name
- 1num
- num1

คำสงวน (Reserved word, Keyword)

• คำสงวน

- คือ คำที่ถูกกำหนดให้เป็นไวยากรณ์ของภาษา Python จึงใช้เป็นชื่อตัวแปรไม่ได้

False	break	for	not
None	class	from	or
True	continue	global	pass
__peg_parser__	def	if	raise
and	del	import	return
as	elif	in	try
assert	else	is	while
async	except	lambda	with
await	finally	nonlocal	yield

```

>>>for = 6

Cell In[1], line 1
  for = 6 # It will give error because "for" is keyword and we cannot
    ^
    use as a variable name.
SyntaxError: invalid syntax

```

- ไม่ควรตั้งชื่อตัวแปรซ้ำกับชื่อของ **built-in functions** ต่างๆ เช่น print, data, float, int, numeric, oxphys, array, close, input, open, range, type, write, zeros

คำสงวน (Reserved word, Keyword)

- รูปแบบการตั้งชื่อ

- ถ้าชื่อตัวแปรยาว myvariablename อาจจะสามารปรับให้อ่านง่าย ดังนี้



Pascal case :

MyVariableName = 20

Camel case :

myVariableName = 20



Snake case :

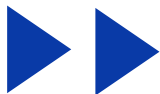
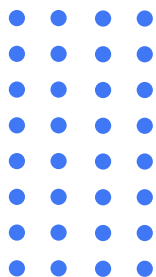
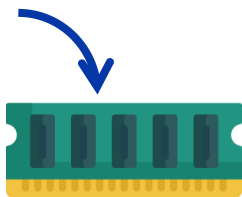
my_variable_name = 20

 Question #2.1



Question #2.2

การประกาศตัวแปร



การประกาศตัวแปร

- ใน Python **ไม่ต้องกำหนดชนิดของตัวแปร** เหมือนภาษาอื่น เช่น ภาษา C ภาษา Java โดย Python จะรู้จากข้อมูล
- **การประกาศตัวแปร:**

ชื่อตัวแปร = ค่าของตัวแปร

```
width = 10    # จำนวนเต็ม  
height = 9.1  # จำนวนจริง  
text = "Hello Python" # ข้อความ (String)
```

```
first_name = 'Pognsathon'  
last_name = 'Janyoi'  
country = 'Thailand'  
city = 'Khon Khaen'  
age = 34  
  
print('First name:', first_name)  
print('Last name: ', last_name)
```

> Output

```
First name: Pognsathon  
Last name: Janyoi
```

การประกาศตัวแปร

- ประกาศตัวแปรหลายตัวแปรในหนึ่งบรรทัด สามารถคั่นด้วย , (comma) และ ; (semicolon)

```
first_name, last_name, age = "Pongsathon", "Janyoi", 34
```

```
print(first_name)  
print(last_name)  
print(age)
```

> Output

```
Pognsathon  
Janyoi  
34
```

```
a = 10; b = 20; c = 30
```

```
print(a,b,c)
```

> Output

```
12 20 30
```

การประกาศตัวแปร

- ประกาศให้หลายๆ ตัวแปรมีค่าเท่ากัน

```
x = y = z = 1.25
```

```
print(x)
```

```
print(y)
```

```
print(z)
```

Output

1.25

1.25

1.25

การกำหนดค่าของตัวแปร

- การกำหนดค่าของตัวแปร ก็สามารถทำได้ในลักษณะเดียวกันกับการประกาศตัวแปร

ชื่อตัวแปร = ค่า หรือ นิพจน์ ที่ต้องการกำหนด

```
name=None  
age=0
```

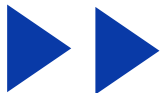
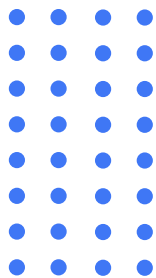
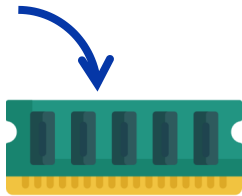
```
print(name)  
print(age)
```

```
new_age=age #นำค่าจากตัวแปร age ไปใส่ใน new_age  
print(new_age)  
new_age = new_age + 5  
print(new_age)  
print(age) #นำค่าของ age จะไม่เปลี่ยนไปเพราะนำค่าไปใช้เท่านั้น
```

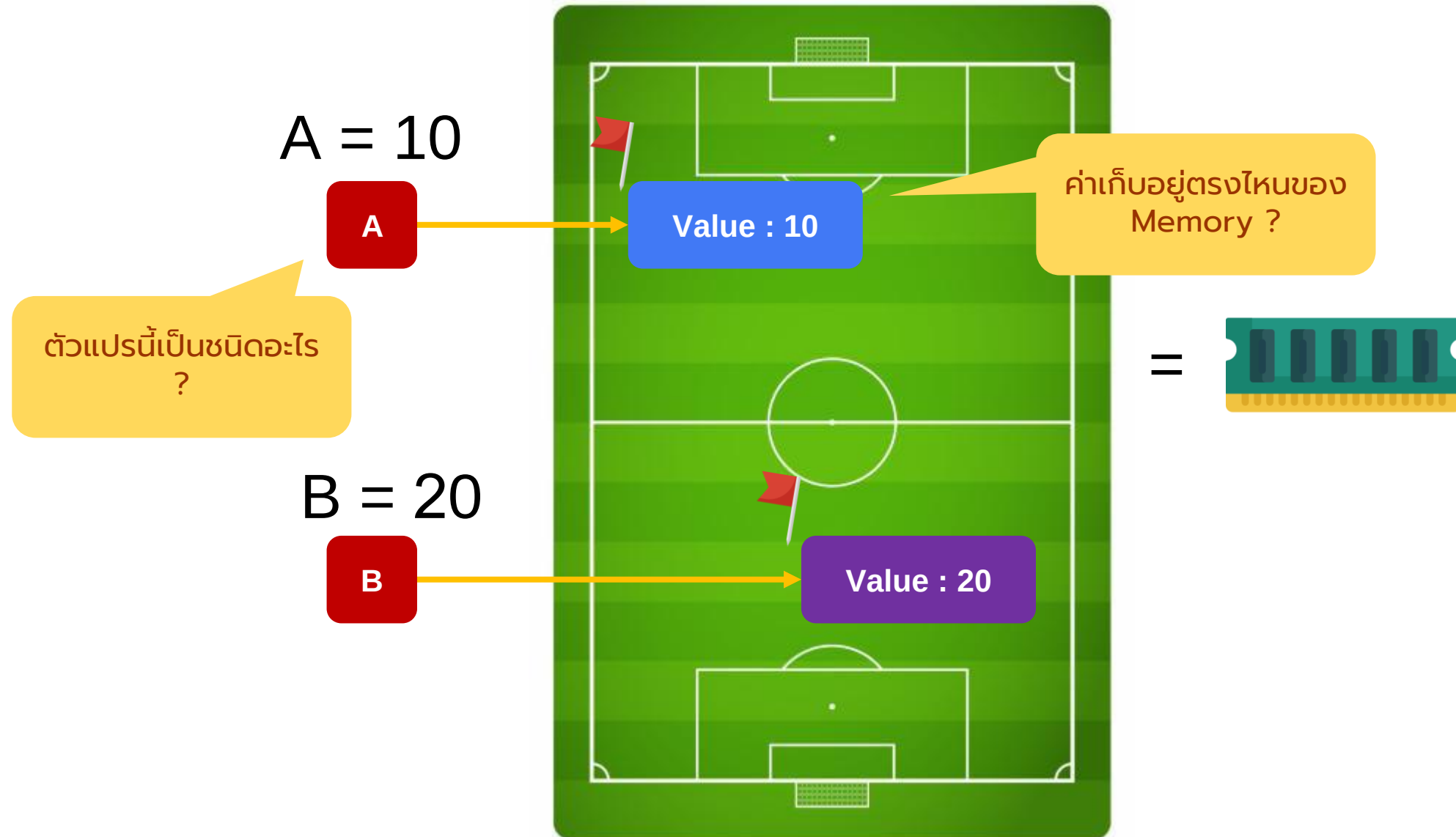
```
> Output  
0  
5  
0
```

 Question #2.3

การตรวจสอบประเภท ของตัวแปร



การตรวจสอบประเภทของตัวแปร



การตรวจสอบชนิดของตัวแปร

- ใช้ฟังก์ชัน **type()** เพื่อดูชนิดของตัวแปร ซึ่งเป็น built-in function
- ใช้ฟังก์ชัน **id()** เพื่อดูตำแหน่งของ object ในหน่วยความจำได้ด้วย

```
age=5
new_age=age
print(id(age),type(age),age)
print(id(new_age),type(new_age), new_age)

new_age = new_age + 0.5
print(id(new_age),type(new_age),new_age)
print(id(age),type(age),age)
```

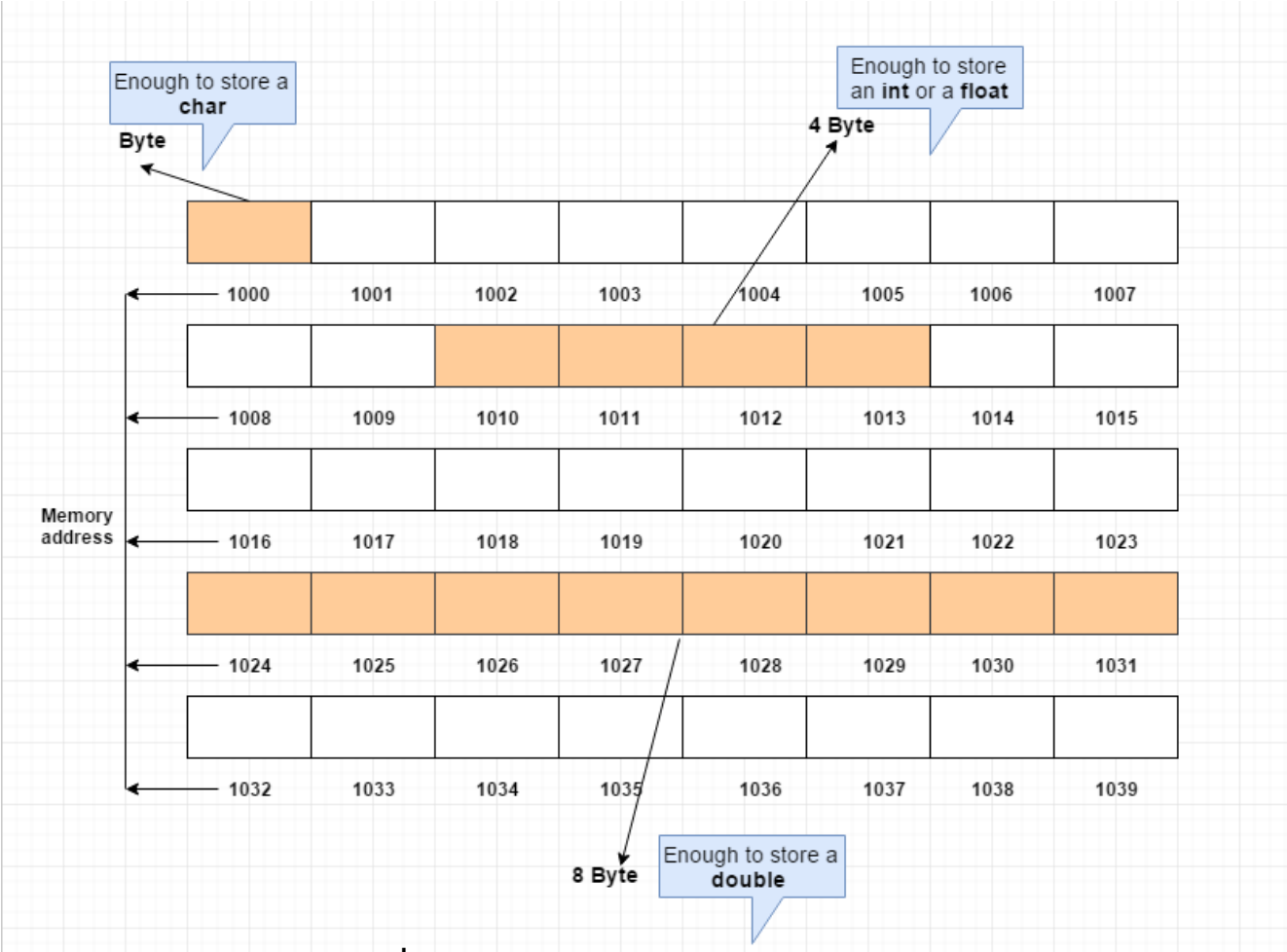
> Output

```
139834892353904 <class 'int'> 5
139834403475568 <class 'float'> 5.5
```

ตำแหน่งในหน่วยความจำ
จะเปลี่ยนไป

ชนิดของตัวแปรจะ
เปลี่ยนไป

ขนาดของตัวแปรในหน่วยความจำ



- ภาษาโปรแกรมอื่นๆ เช่น C หรือ C++
- ขนาดของตัวแปรจะถูกกำหนดชัดเจน เช่น int อาจมีขนาด 4 ไบต์ และ double มีขนาด 8 ไบต์



ขนาดของตัวแปรในภาษา Python

- ภาษา Python หน่วยความจำจะมีการจัดการโดยอัตโนมัติ (**Automatic Memory Management**)
- ผู้เขียนโปรแกรมไม่จำเป็นต้องกำหนดขนาดตัวแปรเอง โดยชนิดข้อมูล เช่น int ใน Python สามารถขยายขนาดได้ตามค่าที่เก็บ
- การตรวจสอบขนาดของตัวแปรสามารถใช้ฟังก์ชัน **getsizeof()** ของโมดูล sys

```
import sys

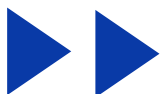
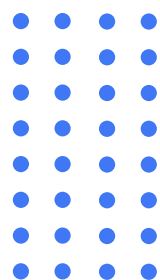
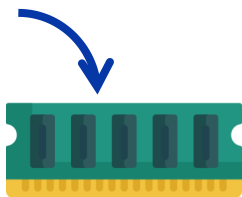
num1 = 100
num2 = 3.14
text1 = "Hello"
list1 = [1, 2, 3]

print('Size of num1:', sys.getsizeof(num1), 'bytes')
print('Size of num2:', sys.getsizeof(num2), 'bytes')
print('Size of text1:', sys.getsizeof(text1), 'bytes')
print('Size of list1:', sys.getsizeof(list1), 'bytes')
```

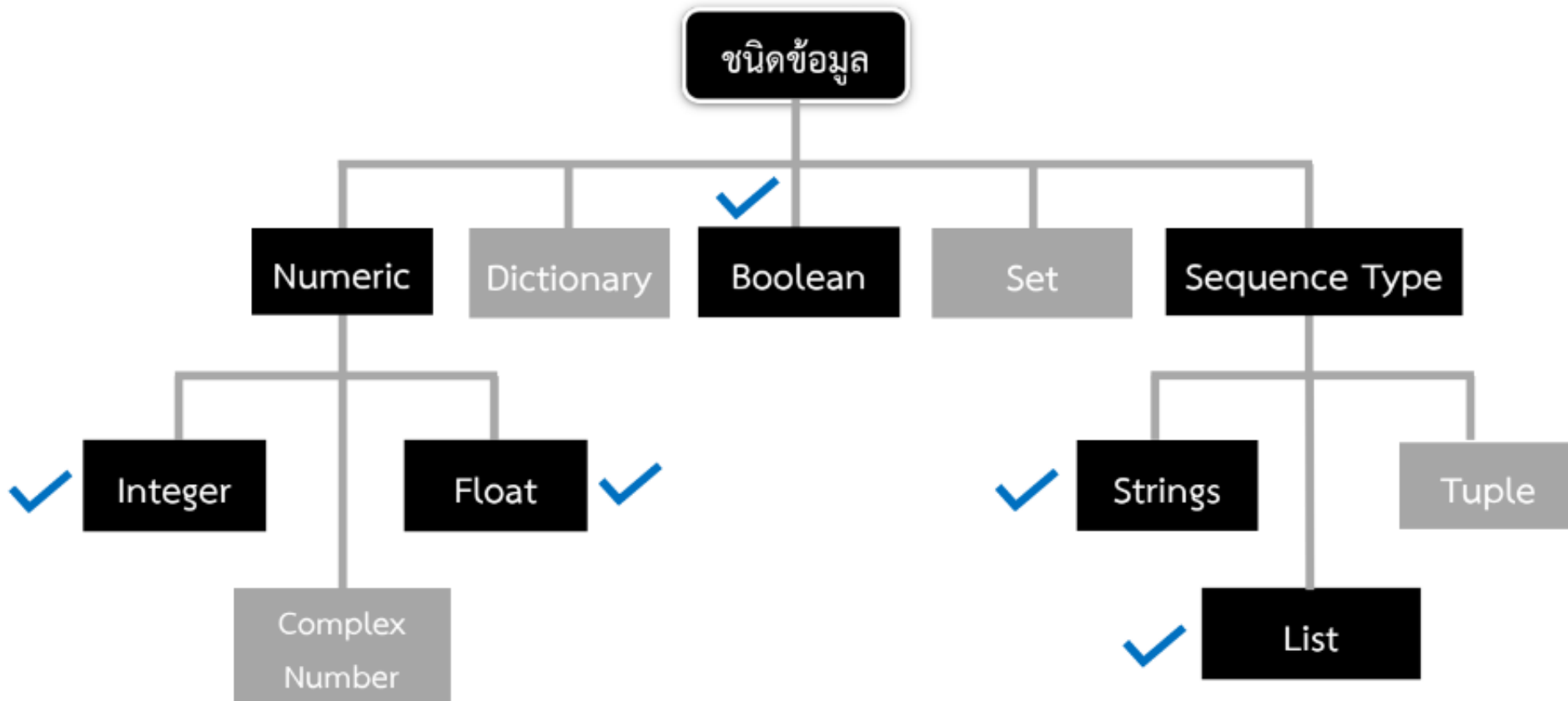
➤ Output

```
Size of num1: 28 bytes
Size of num2: 24 bytes
Size of text1: 46 bytes
Size of list1: 88 bytes
```

ชนิดข้อมูลใน Python (Python Datatypes)



ชนิดข้อมูล (Data Types)

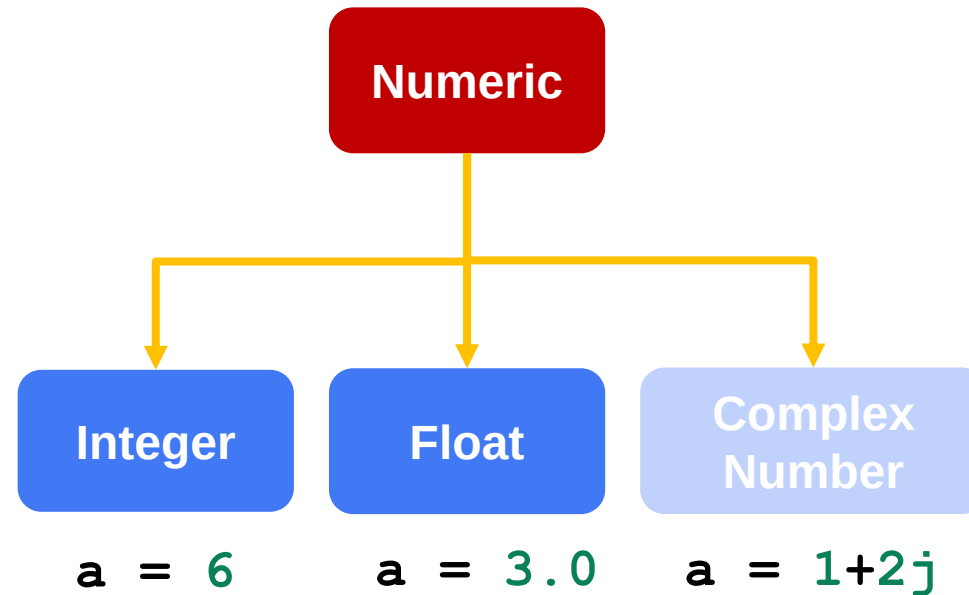


ชนิดของตัวแปรใน Python

- ชนิดข้อมูลพื้นฐาน (Basic data types):
 - เก็บค่าของตัวข้อมูลเองซึ่งมีเพียงหนึ่งค่า ประกอบด้วย ข้อมูลตัวเลข (Numeric) และข้อมูลข้อความ (String)
- ชนิดข้อมูลเชิงประกอบ (Composite data types):
 - เป็นชนิดของตัวแปรกลุ่มที่ผสมหรือประกอบด้วยข้อมูลชนิดต่างๆ ผสมกัน ซึ่งชนิดของข้อมูลอาจจะไม่เหมือนกันก็ได้ ประกอบด้วย List, Tuple, Dictionary และ Set

Python Numbers

- ข้อมูลชนิดที่สามารถนำมาเปลี่ยนแปลงค่าหรือคำนวณทางคณิตศาสตร์ได้



Python Numbers

• การตรวจสอบชนิดตัวแปร

- **type()** - ตรวจสอบชนิดตัวแปร
- **isinstance()** - ตรวจสอบคลาสของตัวแปรตรงกับคลาสที่ต้องการตรวจสอบหรือไม่

ตรวจสอบตัวแปร a
เป็นชนิด int หรือไม่ ?

```
num = 8
print(num, "is of type", type(num))
print(num, "is integer number?", isinstance(num,int))

a = 3.0
print(num, "is of type", type(num))
print(a, "is float number?", isinstance(num,float))

num = 1+2j # '1' is real part and '2j' is imaginary part
print(num, "is of type", type(num))
print(num, "is complex number?", isinstance(num,complex))
```

> Output

```
6 is of type <class 'int'>
6 is integer number? True
3.0 is of type <class 'float'>
3.0 is float number? True
(1+2j) is of type <class 'complex'>
(1+2j) is complex number? True
```

Python Numbers

- ตัวแปรชนิดตัวเลขจำนวนเต็ม (integer)
 - ตัวเลขใน Python มีความยาวไม่จำกัดขึ้นอยู่กับขนาดหน่วยความจำที่เหลืออยู่

ยาวยาว ได้อีก เยอะ

```
a = 12345678901234567890123456789
print ('a=', a)
```

> Output

```
a= 12345678901234567890123456789
```

Python Numbers

- ตัวแปรชนิดตัวเลขทศนิยม (float)

- จำนวนเต็มกับตัวเลขทศนิยมจะถูกแยกแยะโดยจุด (Decimal point) “.”

```
b = 0.1234567890123456789123
print ('b=', b)
```

```
x = 35e3
y = 35e-5
print ('x=', x)
print ('y=', y)
```

กำหนดค่าแบบนี้ก็ได้

- $35e3 = 35 \times 10^3$
- $35e-5 = 35 \times 10^{-5}$

Output

```
b= 0.12345678901234568
x= 35000.0
y= 0.00035
```

เลขหลังจุดเก็บได้ 15 ตัวเท่านั้นนะ



Question #2.4

การแปลงชนิดข้อมูล (casting)

- **int()** – แปลงเป็นตัวเลขจำนวนเต็ม
- **float()** - แปลงเป็นตัวเลขทศนิยม
- **str()** – แปลงตัวเลขเป็นข้อความ

```
num1 = "10"  
num2 = "20.52"
```

```
result = num1+num2  
result_text = 'result = '+result  
print(result_text)
```

ต้องการบวกเลข
ต้องแปลงข้อมูลตรงไหน?

> Output

1020.52

การแปลงชนิดข้อมูล (casting)

- **int()** – แปลงเป็นตัวเลขจำนวนเต็ม
- **float()** - แปลงเป็นตัวเลขทศนิยม
- **str()** - แปลงเป็นข้อความ



```
num1 = int("10")
num2 = int("20.52")

result = num1+num2
print(type(result))
result_text = 'result = '+result
print(result_text)
```

แปลงเป็น int

Output

```
<ipython-input-2-232a91ea98b4> in <cell line: 2>()
      1 num1 = int("10")
----> 2 num2 = int("20.52")
      3
      4 result = num1+num2
      5 print(type(result))

ValueError: invalid literal for int() with base 10: '20.52'
```

"20.52" แปลงเป็น int
ไม่ได้ต้องใช้ float()

การแปลงชนิดข้อมูล (casting)

- **int()** – แปลงเป็นตัวเลขจำนวนเต็ม
- **float()** - แปลงเป็นตัวเลขทศนิยม
- **str()** - แปลงเป็นข้อความ



```
num1 = int("10")
num2 = float("20.52")
```

แปลงเป็น float

```
result = num1+num2
print(type(result))
result_text = 'result = '+result
print(result_text)
```

Output

```
<ipython-input-3-2be8cf087dab> in <cell line: 6>()
      4 result = num1+num2
      5 print(type(result))
----> 6 result_text = 'result = '+result
      7 print(result_text)
```

TypeError: can only concatenate str (not "float") to str

String จะบวกกับ float ไม่ได้

การแปลงชนิดข้อมูล (casting)

- **int()** – แปลงเป็นตัวเลขจำนวนเต็ม
- **float()** - แปลงเป็นตัวเลขทศนิยม
- **str()** - แปลงเป็นข้อความ

```
num1 = int("10")
num2 = float("20.52")

result = num1+num2
print(type(result))
result_text = 'result = ' + str(result)
print(result_text)
```

แปลงเป็น string



> Output

result = 30.25

 Question #2.5

Python Strings

- String เป็นข้อมูลสายอักขระหรือมักนิยมเรียกทับศัพท์ว่า **สตริง**

```
s = 'Python'
print(s)
s = "Python"
print(s)
```

ครอบด้วย ' ' (single quotes)
หรือ " " (double quotes)

```
s = '''I love Python
This is
multiline
string'''
print(s)
```

```
s = """I love Python
This is
multiline
string"""
print(s)
```

ครอบด้วย ''' (triple quotes) หรือ
""" (triple double quotes)

 Output

```
I love Python
This is
multiline
string
```

Python Strings

- ตัวแปรชนิดสตริงไม่สามารถแก้ไขข้อมูลบางส่วนได้ (immutable variable)

```
a = "banona"
a[3] = 'a'
```

```
<ipython-input-13-7427274dad22> in <cell line: 3>()
      1 a = "banona"
      2
----> 3 a[3]='a'
      4
      5 # Simiar to TUPLE, STRING is immutable

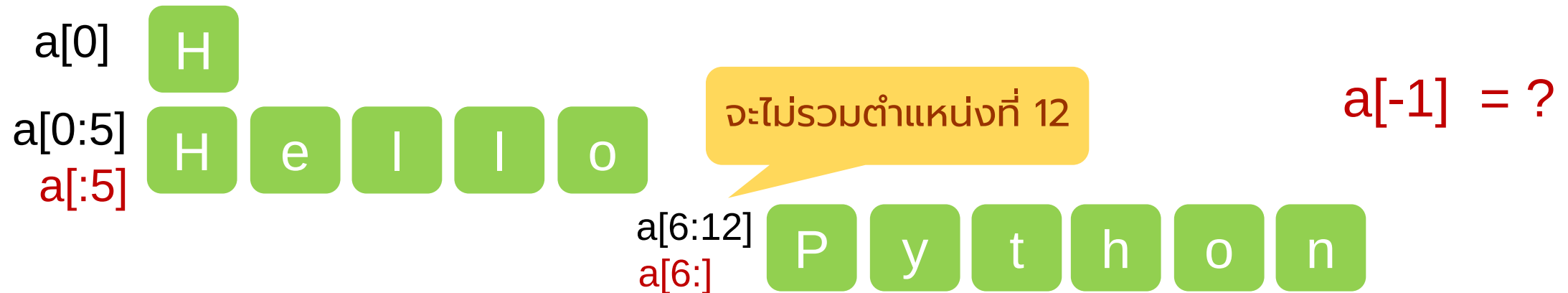
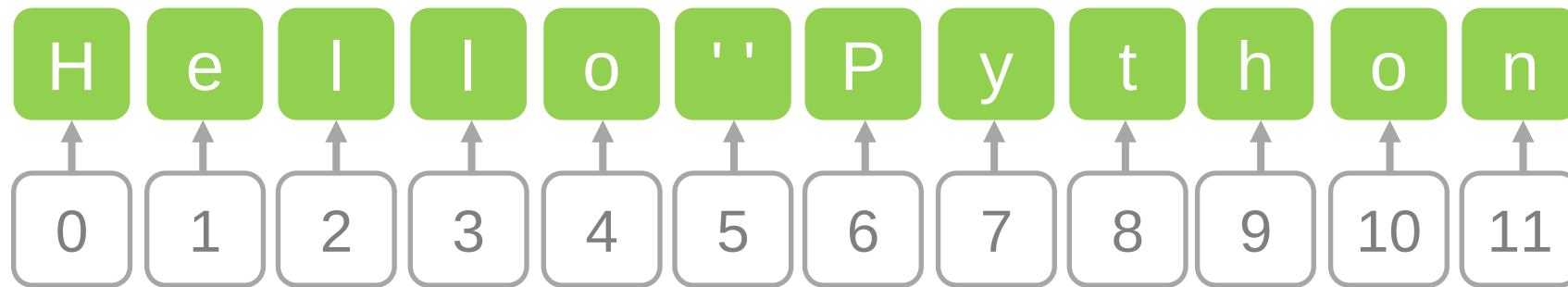
TypeError: 'str' object does not support item assignment
```



Python Strings

- การตัดแบ่งข้อความ (Slicing Strings)

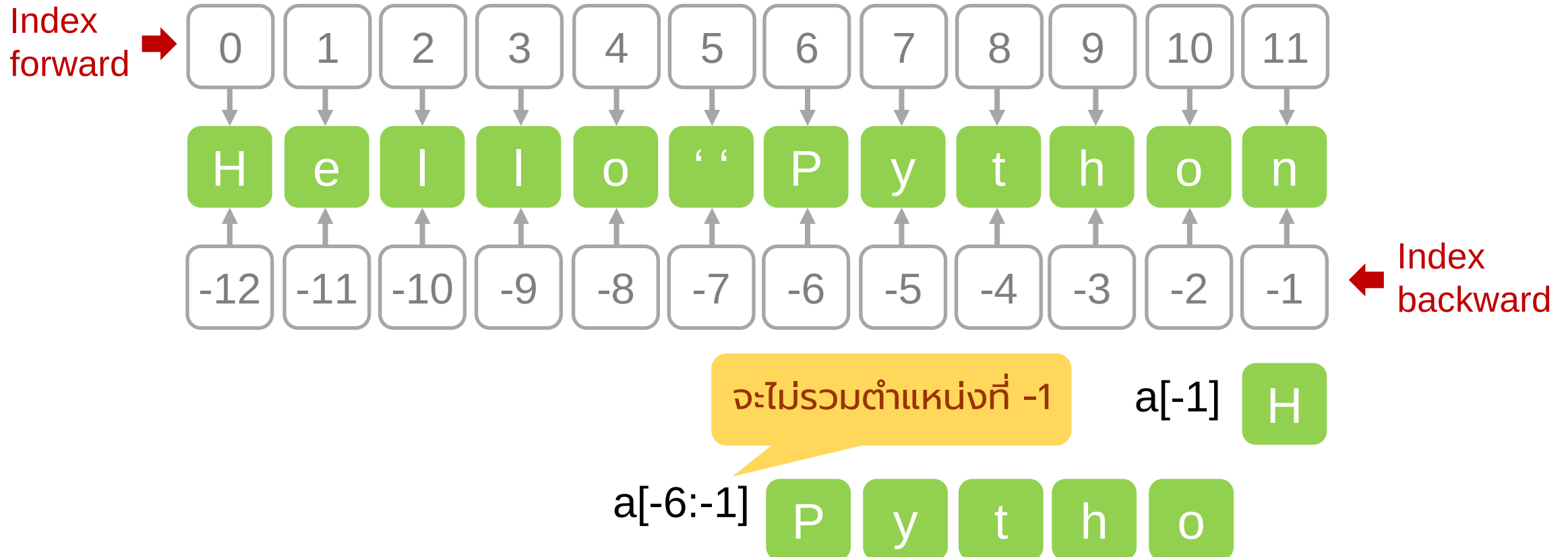
`a = "Hello Python"`



Python Strings

- การตัดแบ่งข้อความ (Slicing Strings)

`a = "Hello Python"`



 Question #2.6

 Question #2.7

เทร็ดสำหรับการจัดการข้อความ (String methods)

- **lower()** : แปลงเป็นตัวอักษรตัวเล็กทั้งหมด
- **upper()** : แปลงเป็นตัวอักษรตัวใหญ่ทั้งหมด
- **strip()** : ลบช่องว่างด้านหน้าและด้านหลังข้อความออก
- **replace()** : แทนที่ข้อความใดๆ ด้วยข้อความใหม่ที่ต้องการ
- **split()** : แบ่งข้อความเป็น List โดยกำหนดตัวแบ่งที่ต้องการ
- **count()** : นับว่าข้อความปรากฏข้อความที่ต้องการนับกี่ครั้ง

เทร็ดสำหรับการจัดการข้อความ (String methods)

```
text = " Hello Python! "  
  
print(len(text))  
print(text.lower())  
print(text.upper())  
print(text.strip())  
#replace  
print(text.replace('Hello', 'Hi, '))  
#split  
print(text.split(" "))  
#strip -> split  
print(text.strip().split(" "))  
#count 'o' in this text  
print(text.count('o'))
```

> Output

```
15  
hello python!  
HELLO PYTHON!  
Hello Python!  
Hi, Python!  
['', 'Hello', 'Python!', '']  
['Hello', 'Python!']  
2
```

การจัดรูปแบบข้อความ (String formatting)

- ใช้ตัวดำเนินการ % :

```
name = "Many"  
age = 25  
print("My name is %s and I am %d years old." % (name, age))
```



 Output

My name is Many and I am 25 years old.

Number formatting table :

<https://www.pythoncheatsheet.org/cheatsheet/string-formatting>

การจัดรูปแบบข้อความ (String formatting)

- ใช้เมธอด format ของคลาส String:

```
name = "Many"  
age = 30  
print("My name is {} and I am {} years old.".format(name, age))
```



 Output

My name is Many and I am 30 years old.

Number formatting table :

<https://www.pythoncheatsheet.org/cheatsheet/string-formatting>

การจัดรูปแบบข้อความ (String formatting)

- ใช้เมธอด format ของคลาส String:

```
pi = 3.141592653589793  
print("The value of pi is approximately {:.2f}".format(pi))
```



 Output

The value of pi is approximately 3.14

Number formatting table :

<https://www.pythoncheatsheet.org/cheatsheet/string-formatting>

การจัดรูปแบบข้อความ (String formatting)

- ใช้เมธอด format ของคลาส String:

```
product = "Banana" ; price = 0.99; qty = 10
total = price * qty
print("{:<10s}{:>6.2f}{:>4d}{:>8.2f}".format(product, price, qty, total))
print("{:<10s}{:>6.2f}{:>4d}{:>8.2f}".format("Red Apple", price, qty, total))
```

> Output

```
Banana    0.99 10  9.90
Red Apple 0.99 10  9.90
```

Number formatting table :

<https://www.pythoncheatsheet.org/cheatsheet/string-formatting>

การจัดรูปแบบข้อความ (String formatting)

- ใช้เมธอด format ของคลาส String:

```
name = "Many"  
age = 35  
print(f"My name is {name} and I am {age} years old.")
```

> Output

My name is Many and I am 35 years old.

Number formatting table :

<https://www.pythoncheatsheet.org/cheatsheet/string-formatting>

Python List []

- List เป็นชนิดของตัวแปรที่สามารถเก็บข้อมูลได้หลายค่า และไม่จำเป็นต้องเป็นข้อมูลชนิดเดียวกัน

```
a = [] #empty list
b = [1, 3.3, 'Python']
```

ประกาศโดยใช้ [] (square brackets) และคั่นสมาชิกแต่ละตัวด้วย , (comma)

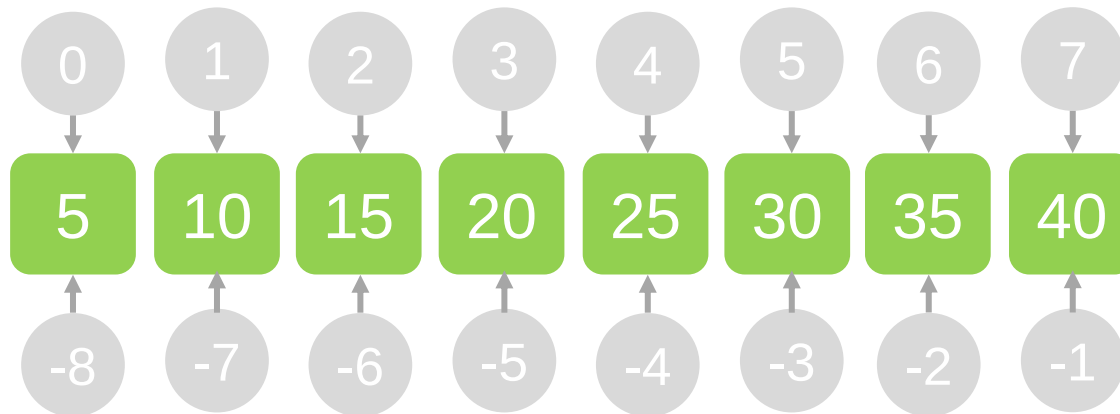
เก็บตัวเลข ข้อความ ผสมกันได้ ไม่ต้องชนิดเดียวกันก็ได้

```
a = [] #empty list
num1 = 1
lang = "Python"
b = [num1, 3.3, lang]
```

Python List [] : การแบ่ง List

- ใช้ [] (slicing operator) เพื่อเข้าถึงข้อมูลเพียงบางส่วนของ List ได้เหมือนกับสตริง

`a = [5, 10, 15, 20, 25, 30, 35, 40]`



`a[2]`

15

`a[-1]`

40

`a[0:3]`

5

10

15

`a[-2:]`

35

40

`a[5:]`

30

35

40

Python List [] : การแบ่ง List

```
a = [5, 10, 15, 20, 25, 30, 35, 40]

print("a[2] = ", a[2])
print("a[0:3] = ", a[0:3])
print("a[5:] = ", a[5:])
print("a[-1] = ", a[-1])
print("a[-2:] = ", a[-2:])
```

> Output

```
a[2] = 15
a[0:3] = [5, 10, 15]
a[5:] = [30, 35, 40]
a[-1] = 40
a[-2:] = [35, 40]
```

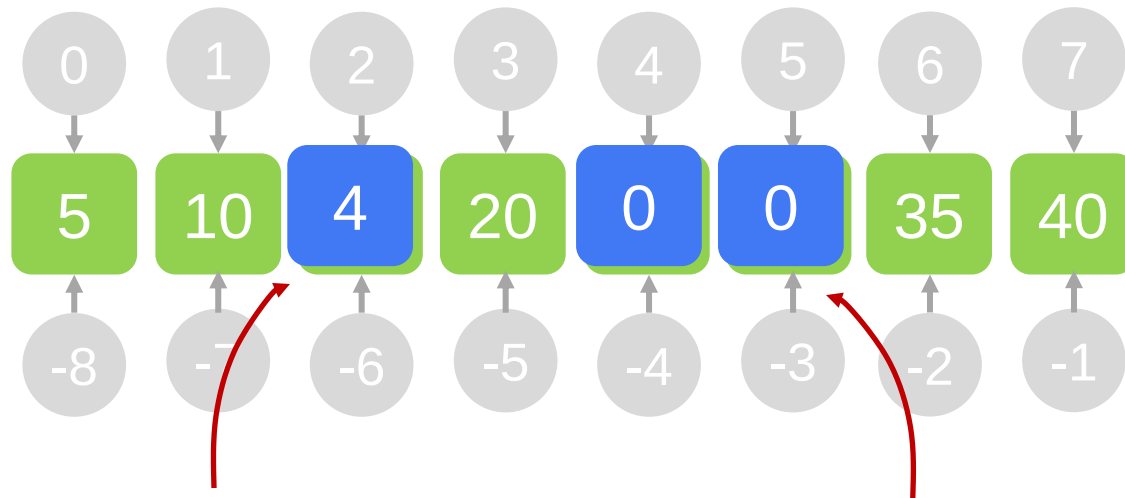
ดึงด้วยตำแหน่งจะได้ค่าข้อมูลที่
เก็บอยู่ตำแหน่งนั้น

ตัดเป็นช่วงจะได้เป็น list

Python List [] : การกำหนดค่า

- การกำหนดค่าภายใน list สามารถกำหนดค่าเดียวหรือเป็นช่วงได้

```
a = [5, 10, 15, 20, 25, 30, 35, 40]
```



```
a[2] = 4
```

```
a[4:6] = [0, 0]
```

Python List [] : แปลง String เป็น list

- สามารถใช้คำสั่ง `split( 'ตัวแบ่ง' )` เพื่อแบ่งข้อความเป็น list

```
animal_text = "Dog Cat Fish"
animals = animal_text.split()
print(animals)
print(len(animals))
```

> Output

```
['Dog', 'Cat', 'Fish']
3
```

```
animal_text = "Dog\tCat\tFish"
animals = animal_text.split('\t')
print(animals)
print(len(animals))
```

> Output

```
['Dog', 'Cat', 'Fish']
3
```

ตัวอย่างการใช้งาน: แปลง String เป็น list

- ตัวอย่าง: การรับค่าชุดตัวเลข หรือตัวเลขหลายตัว

```
nums_text = input("Enter 3 numbers: ")  
nums = nums_text.split(",")
```

```
print('Before convert to int:', nums)  
nums[0] = int(nums[0])  
nums[1] = int(nums[1])  
nums[2] = int(nums[2])  
print('After convert to int:', nums)
```

```
total = nums[0] + nums[1] + nums[2]
```

```
print("Total =", total)
```

 Output

```
Enter 3 numbers: 10,20,30  
Before convert to int: ['10', '20', '30']  
After convert to int: [10, 20, 30]  
Total = 60
```

Python List [] : เมธอดสำหรับการจัดการ list

- **append()** : เพิ่มสมาชิกใน list ต่อท้าย

```
pets = ['Dog', 'Cat', 'Fish']  
pets.append('Hamster')  
print(pets)
```

> Output

```
['Dog', 'Cat', 'Fish', 'Hamster']
```

- **insert()** : แทรกสมาชิกใหม่ลงในตำแหน่งที่กำหนด

```
pets = ['Dog', 'Cat', 'Fish']  
pets.insert(0, 'Parrot')  
pets.insert(2, 'Parrot')  
print(pets)
```

> Output

```
['Parrot', 'Dog', 'Parrot', 'Cat', 'Fish']
```

Python List [] : เมธอดสำหรับการจัดการ list

- **remove() / del / clear()** : ลบสมาชิกออกจาก List ซึ่งจะใช้งาน

แตกต่างกัน

```

pets = ['Dog', 'Cat', 'Fish', 'Dog']
#ลบสมาชิกที่มีค่าเป็น Dog ออกจาก List
pets.remove('Dog')
print(pets)
#ลบตำแหน่ง 0
del pets[0]
print(pets)
#ลบสมาชิกทั้งหมด สามารถกำหนด pets = [] ก็ได้
pets.clear()
print(pets)

```

> Output

```

['Cat', 'Fish', 'Dog']
['Fish', 'Dog']
[]

```

Python List [] : เมร็ดสำหรับการจัดการ list

- สามารถนำ List มาต่อกัน สามารถทำได้โดยบวกกันง่ายๆ

```
pet1 = ['Dog', 'Cat']  
pet2 = ['Fish', 'Dog']  
  
all_pets = pet1 + pet2  
  
print(all_pets)  
print('Number of "Dog":', all_pets.count('Dog'))
```

 Output

```
['Dog', 'Cat', 'Fish', 'Dog']  
Number of "Dog" : 2
```

ตัวอย่างการใช้งาน List

- ตัวอย่าง: การคำนวณค่า Mean Squared Error (MSE)

```
actual_input = input("Enter 3 actual values (separated by comma): ")
predicted_input = input("Enter 3 predicted values (separated by comma): ")
```

```
actual = actual_input.split(",")
predicted = predicted_input.split(",")
```

$$MSE = \frac{(a_1 - p_1)^2 + (a_2 - p_2)^2 + (a_3 - p_3)^2}{3}$$

```
a1 = float(actual[0])
a2 = float(actual[1])
a3 = float(actual[2])
```

```
p1 = float(predicted[0])
p2 = float(predicted[1])
p3 = float(predicted[2])
```

```
error1 = a1 - p1
error2 = a2 - p2
error3 = a3 - p3
```

```
mse=(error1*error1 + error2*error2 + error3*error3)/3
```

```
print("\nMean Squared Error: %.2f" % mse)
```

ตัวอย่างการใช้งาน List

- ตัวอย่าง: การคำนวณค่า Mean Squared Error (MSE)

 Output

Enter 3 actual values (separated by comma): 1,2,4
Enter 3 predicted values (separated by comma): 1,2,0

Mean Squared Error: 5.33

 Output

Enter 3 actual values (separated by comma): 1,2,4
Enter 3 predicted values (separated by comma): 1,2,4

Mean Squared Error: 5.00

Python Tuple ()

- **Tuple** จะเป็นชนิดตัวแปรที่สามารถเก็บข้อมูลได้หลายค่าเหมือนกับ List
 - Tuple จะแก้ไขข้อมูลที่เก็บอยู่ไม่ได้ (Immutable)
 - สร้างมาแล้วจะแก้ไขอะไรไม่ได้

```
t = (6, 'Program Version', 1.02)

print("t[0] = ", t[0])
print("t[1] = ", t[1])
print("t[1:3] = ", t[1:3])
```

```
t[0] = 10
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-50-88963aa635fa> in <cell line: 1>()
----> 1 t[0] = 10
```

```
TypeError: 'tuple' object does not support item assignment
```

> Output

```
t[0] = 6
t[1] = Program Version
t[1:3] = ('Program Version', 1.02)
```

สร้างมาทำไมหลายอัน
ทำไมไม่ใช่ list นี๊



ตัวอย่างการใช้งาน Tuple

• ตัวอย่าง: การเก็บข้อมูล Spam Text และ Label



```
print("Enter 3 email samples")

train_data = []

text1 = input("Email text 1: ")
label1 = input("Label (spam/non-spam): ")
train_data.append((text1, label1))

text2 = input("Email text 2: ")
label2 = input("Label (spam/non-spam): ")
train_data.append((text2, label2))

text3 = input("Email text 3: ")
label3 = input("Label (spam/non-spam): ")
train_data.append((text3, label3))

print("\nTraining Data:")
print(train_data)
```

 Output

```
Enter 3 email samples
Email text 1: Win a free iPhone now! Click this link
Label (spam/ham): spam
Email text 2: Meeting tomorrow at 10 AM in room 301
Label (spam/ham) non-spam
Email text 3: Congratulations! You have won a lottery
prize
Label (spam/ham): spam

Training Data:
[('Win a free iPhone now! Click this link', 'spam'),
('Meeting tomorrow at 10 AM in room 301', 'non-
spam'), ('Congratulations! You have won a lottery
prize', 'spam')]
```

ตัวอย่างการใช้งาน List

- ตัวอย่าง: การคำนวณค่า Mean Squared Error (MSE)

 Output

Enter 3 actual values (separated by comma): 1,2,4
Enter 3 predicted values (separated by comma): 1,2,0

Mean Squared Error: 5.33

 Output

Enter 3 actual values (separated by comma): 1,2,4
Enter 3 predicted values (separated by comma): 1,2,4

Mean Squared Error: 5.00

Python Set {}

- **Set** คือ ตัวแปรที่สามารถเก็บข้อมูลได้หลายตัวไม่เรียงลำดับ แต่สมาชิกจะไม่ซ้ำกัน

```
a = {1, 1, 3, 6, 9}
print(a)
b = {}
print(b)
```

ประกาศโดยใช้วงเล็บปีกกา (braces) { } และคั่นสมาชิกแต่ละตัวด้วย , (comma)

> Output

```
{1, 3, 6, 9}
set()
```

จะไม่มีค่าที่ซ้ำกัน

Python Set {}

- ดำเนินการเกี่ยวกับ set กับตัวแปรชนิดนี้ได้ เช่น union, intersection

```
a = {0,1,2,2,3,3,3,4,4,4,4}
print(a)
b = {1,2,3,4,5,6}
print(b)
print('a union b :',a|b) #a union b
print('a intersect b :',a&b) #a intersect b
print('a difference b :',a-b) #a difference b
print('a symmetric difference b :',a^b) #a
symmetric difference b
```

> Output

```
{0, 1, 2, 3, 4}
{1, 2, 3, 4, 5, 6}
a union b : {0, 1, 2, 3, 4, 5, 6}
a intersect b : {1, 2, 3, 4}
a difference b : {0}
a symmetric difference b : {0, 5, 6}
```

Python Set {}

- เมธอด **add()** ใช้สำหรับเพิ่มสมาชิกในตัวแปร set
- เมธอด **remove()** ใช้สำหรับลบสมาชิกในตัวแปร set

```
a = {1, 2, 3}
```

```
a.add(4)
```

```
print(a)
```

```
a.remove(1)
```

```
print(a)
```

Output

```
{1, 2, 3, 4}
```

```
{2, 3, 4}
```

Python Set {}

- เมธอด **pop()** ใช้สำหรับอ่านค่าตัวแรกในตัวแปร set แล้วลบสมาชิกตัวนั้นออก
- เมธอด **clear()** ใช้สำหรับลบสมาชิกทั้งหมดในตัวแปร set

```
a = {0, 1, 2, 3, 4}
b = a.pop()
print(a)
print(b)
a.clear()
print(a)
```

Output

```
{1, 2, 3, 4}
0
set()
```

ตัวอย่างการใช้งาน Set

- ตัวอย่าง: การสร้างรายการคำที่ไม่ซ้ำ (Unique Vocabulary)

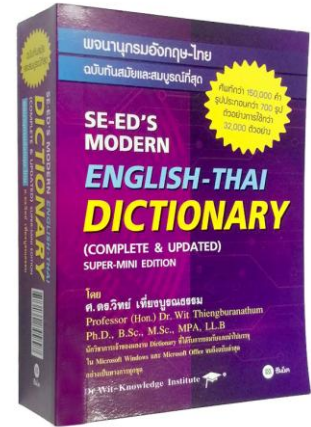
```
text = input("Enter a sentence: ")  
  
words = text.split()  
  
unique_words = set(words)  
  
print("Unique words:")  
print(unique_words)
```

>- Output

```
Enter a sentence: machine learning is fun and  
machine learning is useful  
Unique words:  
{'is', 'and', 'learning', 'useful', 'machine', 'fun'}
```

Python Dictionary {key: value}

- **Dictionary** เป็นตัวแปรแบบที่เก็บข้อมูลลักษณะแบบเป็น
 - มีคีย์ (Key) ซึ่งเป็นตัวระบุค่า (Value)
 - คีย์จะไม่ซ้ำกัน คีย์สามารถเป็นได้ทั้งตัวเลข และความ



Key

Value

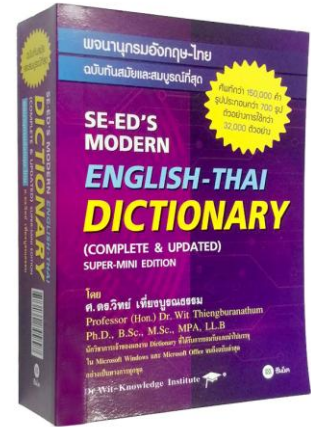
```
animals = {'Dog': 'สุนัข', 'Cat': 'แมว', 'Ant': 'มด'}
print(animals)
print(type(animals))
print("animals['Cat'] = ", animals['Cat'])
print("animals['Dog'] = ", animals['Dog'])
```

> Output

```
{'Dog': 'สุนัข', 'Cat': 'แมว', 'Ant': 'มด'}
<class 'dict'>
animals['Cat'] = แมว
animals['Dog'] = สุนัข
```

Python Dictionary {key: value}

- **Dictionary** เป็นตัวแปรแบบที่เก็บข้อมูลลักษณะแบบเป็นคู่ (key-value)
 - คีย์จะไม่ซ้ำกัน คีย์สามารถเป็นได้ทั้งตัวเลข และข้อความ



จำนวนเต็ม

ทศนิยม

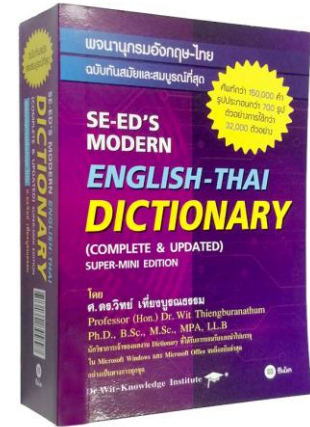
```
grade = {4: 'A', 3.5 : 'B+', 3 : 'B', 2.5: 'C+', 2: 'C', 1.5: 'D+', 1: 'D', 0: 'F'}
print(grade)
print("grade[4] = ", grade[4])
print("grade[3.5] = ", grade[3.5])
```

> Output

```
{4: 'A', 3.5: 'B+', 3: 'B', 2.5: 'C+', 2: 'C', 1.5: 'D+', 1: 'D', 0: 'F'}
grade[4] = A
grade[3.5] = B+
```

Python Dictionary {key: value}

- **Dictionary** เป็นตัวแปรแบบที่เก็บข้อมูลลักษณะแบบเป็น
 - จะต้องอ้างอิงคีย์ที่มีอยู่ใน Dictionary เท่านั้น



```
animals = {'Dog': 'สุนัข', 'Cat' : 'แมว', 'Ant': 'มด'}
print(animals)
print(type(animals))
print("animals['Cat'] = ", animals['cat'])
print("animals['Dog'] = ", animals['dog'])
```

Output

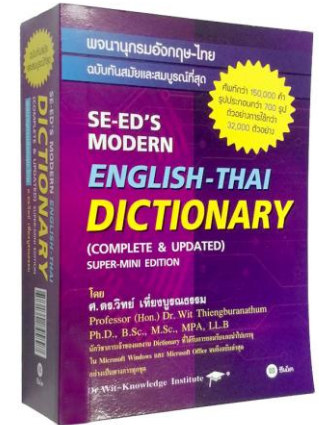
```
Traceback (most recent call last):
  File "D:\python\hello.py", line 120, in <module>
    print("animals['Cat'] = ", animals['cat'])
    ~~~~~^~~~~~
KeyError: 'cat'
```

Python Dictionary {key: value}

- สามารถกำหนดค่าโดยอ้างอิงด้วยคีย์ (Key)

```
emp = {'id': '0001',
      'name': 'Many Jany',
      'age': 30,
      'salary': 200}

print("old salary =", emp['salary'])
emp['salary'] = 50000 #กำหนด salary
print("new salary =", emp['salary'])
```

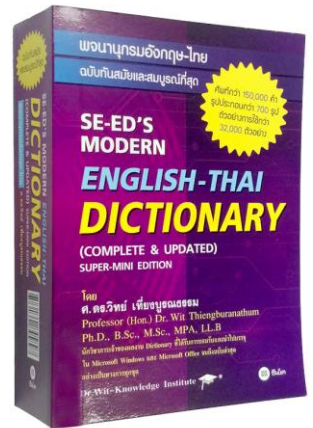


> Output

```
old salary = 200
new salary = 50000
```

Python Dictionary {key: value}

- กรณีไม่ได้ระบุ Key ไว้ตอนประกาศตัวแปรสามารถเพิ่มได้ใน



```
emp = {'id': '0001',  
      'name': 'Many Jany',  
      'age': 30}
```

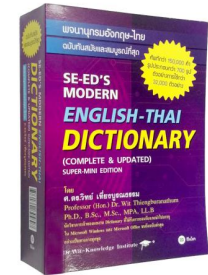
```
emp['phone'] = '09-4693-76xx'  
print(emp)
```

ไม่ได้ใส่ คีย์ "phone"
ตอนประกาศ

> Output

```
{'id': '0001', 'name': 'Many Jany', 'age': 30, 'phone':  
'09-4693-76xx'}
```

Python Dictionary {key: value}



- เมทออดสำหรับการจัดการตัวแปรชนิด Dictionary
- **keys()** ใช้สำหรับดึงคีย์ทั้งหมดของ Dictionary
- **values()** ใช้สำหรับดึงค่าทั้งหมดของ Dictionary
- **items()** ใช้สำหรับดึงข้อมูลทั้งหมดที่อยู่ใน Dict เป็นรายการแบบ tuple(key, values)

```
emp = {'id': '0001',
      'name': 'Many Jany', 'age': 30, 'sex': None}
keys = emp.keys()
print('keys:', tuple(keys)) #แปลงเป็น tuple ก่อน
print
values = emp.values()
print('values:', tuple(values)) #แปลงเป็น tuple ก่อน
print
items = emp.items()
print('items:', list(items)) #แปลงเป็น list ก่อน print
```

> Output

```
keys : ('id', 'name', 'age', 'sex')
values : ('0001', 'Many Jany', 30, None)
items : [('id', '0001'), ('name', 'Many Jany'), ('age', 30),
         ('sex', None)]
```

A decorative element consisting of a grid of blue dots arranged in two columns of ten dots each.

Question #2.8



Question #2.9

A decorative element consisting of a grid of blue dots arranged in two columns of ten dots each.

Question #2.10

List + Dicts

list

```
customer_list = [  
    {  
        'name': 'John Doe',  
        'age': 35,  
        'email': 'johndoe@example.com'  
    },  
    {  
        'name': 'Jane Smith',  
        'age': 28,  
        'email': 'janesmith@example.com'  
    },  
    {  
        'name': 'Michael Johnson',  
        'age': 42,  
        'email': 'michaeljohnson@example.com'  
    }  
]
```

dict

ทบทวนท้ายบท

- การประกาศตัวแปร การตั้งชื่อ คำสงวน
- การตรวจสอบชนิดของตัวแปร `type()` ตำแหน่งในหน่วยความจำ `id()`
- ชนิดข้อมูลพื้นฐาน
 - ตัวแปรชนิดตัวเลข : `int`, `float`, `complex`
 - ตัวแปรชนิดข้อความ
- การแปลงชนิดตัวแปร (Casting)
- การตัดแบ่งข้อความ (Slicing)
 - การจัดการข้อความ, การจัดรูปแบบข้อความ
- ชนิดข้อมูลเชิงประกอบ
 - `List`, `Tuple`, `Set`, `Dictionary`